RECOMMENDATION SYSTEM USING DSA

Personalized Suggestions through DSA

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

Salman Ahmed (2520089070)

P Venkata Harshavardhan (2520089071)

Team Number:22

Section: 11

Under the guidance of

<<Dr Anitha Chekuri>>

<<Data Structures And Algorithms>>



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "**Recommendation System using DSA**" is a record of Bonafide work of **Salman Ahmed – 2520089070, P Venkata Harshavardhan – 2520089071**submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Salman Ahmed – 2520089070

P Venkata Harshavardhan – 2520089071

K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the mini project based report entitled "**Recommendation System using DSA**" is a bonafide work done and submitted by **Salman Ahmed – 2520089070, P Venkata Harshavardhan – 2520089071**in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year 2026-2027.

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, <<Guide Name>>, <<Designation>>, <Department>> which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Salman Ahmed – 2520089070

P Venkata Harshavardhan – 2520089071

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	14
8.	Conclusion & Future Scope	16
9.	References	17

1.ABSTRACT

This project presents a Recommendation System built entirely from core Data Structures and Algorithms (DSA). A recommendation system analyses user preferences, behavioural patterns and item metadata to predict and surface the content most relevant to each individual user. The system stores user–item interactions such as ratings, clicks and purchases in hash maps and an inverted index, and models the relationships between users and items as a bipartite graph. Cosine similarity measures how alike two users, or two items, are, and a max-heap (priority queue) retrieves the Top-K neighbours and the Top-N recommendations without sorting the full candidate list. Two strategies are combined: collaborative filtering, which uses nearest-neighbour search over the user–item matrix, and content-based filtering, which compares item feature vectors. The approach is demonstrated on a small movie recommendation example in which a user receives personalised suggestions based on the ratings of similar users. The results show that suitable data structures give O(1) lookups, limit similarity computation to relevant users and extract each recommended item in O(log n) time, which keeps recommendations fast and scalable. The report also discusses benefits and challenges such as cold-start users, sparse data, scalability and algorithmic bias.

Team Name:

Team Logo (if any):

Team Members:

1. Salman Ahmed – 2520089070
2. P Venkata Harshavardhan – 2520089071

2.INTRODUCTION

Online platforms for movies, music, shopping and news hold thousands of items, and no user can browse all of them. A recommendation system is an intelligent algorithm that analyses user preferences, behavioural patterns and item metadata to predict and surface the content most relevant to each individual user. Its core input is a set of user–item interactions such as ratings, clicks and purchase history.

The quality of a recommender depends not only on the prediction idea but also on how efficiently data is stored, searched and ranked. Data Structures and Algorithms make this possible: arrays and hash maps store user profiles and item metadata for O(1) lookups, graphs model user–item relationships for collaborative filtering, heaps retrieve the Top-N recommendations, sorting and searching algorithms rank and retrieve items across large datasets, and tries support scalable search over item names.

Two main strategies are used. Content-based filtering uses item feature vectors and cosine similarity to match items with a user's preferences. Collaborative filtering analyses the user–item interaction matrix with nearest-neighbour algorithms and can use Pearson correlation or matrix factorization to identify latent patterns.

The workflow has two steps. In Step 1 (Collect and Represent Data), interactions are stored in adjacency matrices or graphs: every user and item becomes a node, edges represent relationships and weights reflect interaction strength. In Step 2 (Score, Rank and Deliver Results), similarity scores are computed using cosine similarity or Euclidean distance, a max-heap or priority queue ranks the candidate items, and the Top-K results are displayed as personalised recommendations.

The rest of this report is organised as follows: Section 3 reviews related work, Section 4 states the problem and objectives, Section 5 lists the requirements, Section 6 gives the implementation modules, Section 7 presents results and discussion, Section 8 gives the conclusion and future scope, and Section 9 lists the references.

3.LITERATURE SURVEY

The following similar systems and studies were reviewed for this project. Their sources are listed in the References section.

1. GroupLens (Resnick et al., 1994). An open architecture for collaborative filtering of Usenet news. Readers rate articles, and the system predicts how much a reader will like unseen articles using the ratings of readers with correlated tastes. It is one of the first systems to apply the nearest-neighbour idea used in this project.

2. Amazon item-to-item collaborative filtering (Linden, Smith and York, 2003). Instead of comparing users, the system finds items that customers tend to buy together and builds an offline table of similar items. At request time a recommendation is a quick lookup in that table, which lets the method scale to millions of customers and catalogue items.

3. Item-based collaborative filtering (Sarwar et al., 2001). Studies item–item similarity measures such as cosine, correlation and adjusted cosine, and predicts a rating from the user's ratings of similar items. The work shows that item-based neighbourhoods can give better scalability and quality than user-based ones.

4. Matrix factorization for the Netflix Prize (Koren, Bell and Volinsky, 2009). Describes how users and movies are mapped to latent factors so that a rating is predicted from their product. Factorization models handled sparse rating data well and became the standard strong baseline for movie recommendation.

5. MovieLens datasets (Harper and Konstan, 2015). Explains the history and structure of the MovieLens rating datasets collected from a live movie recommender. They are widely used to test recommendation algorithms and are a natural dataset for extending this project beyond the small demonstration.

6. Survey of recommender systems (Adomavicius and Tuzhilin, 2005). Classifies recommenders into content-based, collaborative and hybrid approaches and lists their limitations, including cold-start users, sparse ratings and limited scalability. These limitations are the challenges discussed in this report.

7. Content-based recommender systems (Lops, de Gemmis and Semeraro, 2011). Reviews how item features are represented as vectors and compared with a user profile using measures such as cosine similarity. This is the basis of the genre-vector similarity used in the movie example.

8. Algorithmic foundations (Cormen et al., Introduction to Algorithms). Standard reference for the hash tables, graphs, heaps and priority queues, sorting and searching techniques used here, including their time-complexity analysis.

4.Problem Statement and Objectives

Problem Statement:

To design an efficient and scalable recommendation system that suggests relevant items to every user by using core data structures and algorithms.

Problem Description:

Platforms hold large and sparse user–item data, and scanning and sorting all of it for every request is slow. Suitable data structures are needed for fast lookup, similarity search and Top-K ranking.

Objectives:

Identify the Project's Purpose:

The project was initiated to show how DSA concepts (hash maps, graphs, heaps, sorting and searching) can be combined into a working personalised recommender that gives fast and relevant suggestions on large datasets. The objectives are:

1. Represent user–item interactions using hash maps, an inverted index and a bipartite graph.

2. Measure similarity between users and between items using cosine similarity.

3. Retrieve the Top-K similar users and Top-N recommendations efficiently using a max-heap (priority queue).

4. Combine collaborative filtering and content-based filtering in a movie recommendation example.

5. Analyse the time complexity of the system and discuss its benefits and challenges (cold start, sparsity, scalability and bias).

5.HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 / AMD Ryzen 3 or higher
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 500 MB of free disk space

Software Requirements:

- Operating System: Windows 10/11 or Linux
- Programming Language: Python 3.10 or later
- IDE / Editor: Visual Studio Code or PyCharm
- Libraries: Python standard library only (heapq, collections, math)

6.IMPLEMENTATION

The system is organised into four modules, written in Python using only the standard library. A small movie dataset of six users and eight movies is used to demonstrate it.

Module 1: Data Representation (hash maps, inverted index, bipartite graph)

```
ratings = {
    "Asha":  {"Inception": 5, "Interstellar": 5, "The Matrix": 4, "Titanic": 2},
    "Bala":  {"Inception": 4, "The Matrix": 5, "Avengers": 4, "Interstellar": 4},
    "Chitra":{"Titanic": 5, "The Notebook": 5, "Inception": 2},
    "Dev":   {"Avengers": 5, "Iron Man": 5, "The Matrix": 3},
    "Esha":  {"Titanic": 4, "The Notebook": 5, "La La Land": 5},
    "Farhan":{"Inception": 5, "Interstellar": 4, "Avengers": 3, "Iron Man": 4},
}
genres = {  # [SciFi, Action, Romance]
    "Inception": [1, 1, 0], "Interstellar": [1, 0, 0], "The Matrix": [1, 1, 0],
    "Avengers": [1, 1, 0], "Iron Man": [1, 1, 0], "Titanic": [0, 0, 1],
    "The Notebook": [0, 0, 1], "La La Land": [0, 0, 1],
}
item_users = defaultdict(set)              # inverted index: movie -> users who rated it
graph = defaultdict(set)                   # bipartite graph (adjacency list)
```

```
for u, items in ratings.items():
    for m in items:
        item_users[m].add(u)
        graph[u].add(m); graph[m].add(u)
```

Module 2: Similarity Measures (cosine similarity)

```
def cosine_ratings(a, b):
    shared = a.keys() & b.keys()
    if not shared: return 0.0
    dot = sum(a[i] * b[i] for i in shared)
    na = math.sqrt(sum(v * v for v in a.values()))
    nb = math.sqrt(sum(v * v for v in b.values()))
    return dot / (na * nb)

def cosine_vec(x, y):
    dot = sum(p * q for p, q in zip(x, y))
    nx = math.sqrt(sum(p * p for p in x)); ny = math.sqrt(sum(q * q for q in y))
    return dot / (nx * ny) if nx and ny else 0.0
```

Module 3: Max-Heap for Top-K Retrieval

```
def top_k(scores, k):
    heap = [(-s, name) for name, s in scores.items()]   # negate: heapq is a min-heap
    heapq.heapify(heap)                                 # O(n)
    out = []
    for _ in range(min(k, len(heap))):                  # each pop is O(log n)
        s, name = heapq.heappop(heap)
        out.append((name, -s))
    return out
```

Module 4: Recommendation Engine and Driver Program

```
def recommend(user, k_neighbors=3, n=3):
    me = ratings[user]
    candidates = {v for m in me for v in item_users[m]} - {user}   # via inverted index
    sims = {v: cosine_ratings(me, ratings[v]) for v in candidates}
    neighbors = top_k(sims, k_neighbors)
    num, den = defaultdict(float), defaultdict(float)
    for v, s in neighbors:
        for m, r in ratings[v].items():
            if m not in me:
                num[m] += s * r; den[m] += s
    return neighbors, top_k({m: num[m] / den[m] for m in num}, n)

def similar_movies(movie, n=3):
    scores = {m: cosine_vec(genres[movie], g) for m, g in genres.items() if m != movie}
    return top_k(scores, n)

if __name__ == "__main__":
    print("Users:", len(ratings), "| Movies:", len(item_users))
    nb, recs = recommend("Asha")
    print("\nNearest neighbours of Asha:")
    for v, s in nb: print(f"  {v:<8} similarity = {s:.3f}")
    print("\nTop recommendations for Asha:")
    for m, s in recs: print(f"  {m:<12} predicted rating = {s:.2f}")
    print("\nMovies similar to 'Inception' (genre cosine):")
    for m, s in similar_movies("Inception"): print(f"  {m:<12} score = {s:.3f}")
    print("\nTop recommendations for Esha:")
    for m, s in recommend("Esha")[1]: print(f"  {m:<12} predicted rating = {s:.2f}")
```

7.RESULTS & DISCUSSION

The program was run on the sample dataset. The console output is shown below (replace with screenshots of your own run if required).

```
Users: 6 | Movies: 8

Nearest neighbours of Asha:
  Bala     similarity = 0.839
  Farhan   similarity = 0.662
  Chitra   similarity = 0.325

Top recommendations for Asha:
  The Notebook predicted rating = 5.00
  Iron Man     predicted rating = 4.00
  Avengers     predicted rating = 3.56

Movies similar to 'Inception' (genre cosine):
  Avengers     score = 1.000
  Iron Man     score = 1.000
  The Matrix   score = 1.000

Top recommendations for Esha:
  Interstellar predicted rating = 5.00
  The Matrix   predicted rating = 4.00
  Inception    predicted rating = 2.41
```

Explanation. The hash maps and inverted index find every user who shares at least one movie with Asha, so only relevant users are compared. Bala (0.839) and Farhan (0.662) are the nearest neighbours because they rate Inception, Interstellar and The Matrix highly, like Asha; the max-heap returns them in order of similarity.

Recommendations. Unseen movies are scored by the similarity-weighted average rating of the neighbours, and the Top-N are taken from the heap. The Notebook receives 5.00 because only one neighbour rated it, which shows how sparse data can give extreme predictions on a small dataset.

Content-based result. Genre-vector cosine similarity gives a score of 1.000 to Avengers, Iron Man and The Matrix for Inception because they share the same genre vector, so related movies are found even without rating data.

Time complexity. Hash-map lookup is O(1). Heapify is O(n) and each extraction is O(log n), so Top-K needs O(n + k log n). For C candidate users with d ratings on average, similarity scoring costs about O(C·d), so each query is much cheaper than comparing against all users and sorting everything.

Benefits. Optimised data structures give faster query resolution. Organised storage using trees, graphs and hash maps keeps the data structured. Graph traversal and similarity algorithms give accurate, personalised results with high precision and reduced latency.

Challenges. Cold-start users lack interaction history, which limits accuracy. Sparse data matrices leave gaps in user–item relationships. Scalability becomes critical as data grows. Algorithmic bias can amplify existing preferences and reduce diversity and fairness.

8.CONCLUSION & FUTURE SCOPE

The project built a recommendation system using only core data structures and algorithms and met its objectives: user–item interactions were represented with hash maps, an inverted index and a bipartite graph; cosine similarity measured user and item closeness; a max-heap retrieved the Top-K neighbours and Top-N recommendations; and collaborative and content-based filtering were combined in a movie example. DSA provides the foundation of a robust, scalable recommender: hash maps give O(1) user–item lookups, graphs model complex relationships, sorting and heaps rank candidates by relevance, and trees support fast search.

Future scope.

Near term: load a real dataset such as MovieLens from files, add Pearson correlation and Euclidean distance, and handle cold-start users with popularity-based suggestions.

Medium term: add matrix factorization, tries for autocomplete search over item names, caching of similarity scores, and dynamic programming for optimisation.

Long term: deploy as a real-time web service with a user interface, use hybrid models that adapt with every interaction, and add diversity and fairness controls to reduce algorithmic bias.

9.REFERENCES

1. J. Resnick, N. Iacovou, M. Suchak, P. Bergstrom and J. Riedl, "GroupLens: An open architecture for collaborative filtering of netnews," Proc. ACM CSCW, 1994. https://doi.org/10.1145/192844.192905
2. G. Linden, B. Smith and J. York, "Amazon.com recommendations: Item-to-item collaborative filtering," IEEE Internet Computing, vol. 7, no. 1, 2003. https://doi.org/10.1109/MIC.2003.1167344
3. B. Sarwar, G. Karypis, J. Konstan and J. Riedl, "Item-based collaborative filtering recommendation algorithms," Proc. WWW, 2001. https://doi.org/10.1145/371920.372071
4. Y. Koren, R. Bell and C. Volinsky, "Matrix factorization techniques for recommender systems," Computer, vol. 42, no. 8, 2009. https://doi.org/10.1109/MC.2009.263
5. F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," ACM Trans. Interactive Intelligent Systems, vol. 5, no. 4, 2015. https://doi.org/10.1145/2827872
6. G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," IEEE TKDE, vol. 17, no. 6, 2005. https://doi.org/10.1109/TKDE.2005.99
7. P. Lops, M. de Gemmis and G. Semeraro, "Content-based recommender systems: State of the art and trends," in Recommender Systems Handbook, Springer, 2011. https://doi.org/10.1007/978-0-387-85820-3_3
8. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press.